

Component-driven architecture

COMPLETE LEARNING GUIDE

What You'll Learn

Every modern frontend framework organizes an application as a tree of components — small, focused, reusable pieces that each own a slice of UI and, ideally, a single responsibility. React expresses this with functional components and JSX, Angular with TypeScript classes and separate template files, and Vue with single-file components combining template/script/style — but the underlying discipline is identical across all three: break the interface into pieces small enough to reason about individually, pass data down through explicit inputs, and compose small components into larger ones. This submodule focuses primarily on React syntax (since it's the most common target in interviews and job postings) while explicitly calling out the equivalent concept in Angular and Vue at each step, so the architectural thinking transfers regardless of which framework a given job uses.

1. What a Component Actually Is

A component is a self-contained, reusable unit that takes inputs and produces UI — conceptually similar to a function that takes arguments and returns a value, except the return value is a description of DOM structure rather than a plain value. In React, a functional component is literally a JavaScript function that returns JSX and accepts a single props object as its argument. Angular components are classes decorated with `@Component`, paired with an HTML template. Vue single-file components (`.vue` files) bundle a `<template>`, a `<script>`, and optionally scoped `<style>` together in one file. All three compile down to instructions for creating and updating real DOM nodes.

JSX

```
function UserCard({ name, role, avatarUrl }) {
  return (
    <div className="user-card">
      <img src={avatarUrl} alt={`${name}'s avatar`} />
      <h3>{name}</h3>
      <p>{role}</p>
    </div>
  );
}

// Usage — data flows in through props, one-directionally
<UserCard name="Aisha" role="Admin" avatarUrl="/avatars/aisha.png" />
```

2. Props: The One-Way Data Contract

Props are how data flows into a component from its parent — read-only from the receiving component's perspective (a component must never mutate its own props directly). This one-directional flow (data down, events up) is what keeps a component tree predictable: to know why a piece of UI looks a certain way, you only need to look at what was passed in, not search the whole app for anything that might have changed it. When a child needs to communicate back up to a parent (a button click, a form submission), the parent passes a callback function down as a prop, and the child calls it — the data itself still only ever flows downward.

- `props.children` (React) / `<ng-content>` (Angular) / `<slot>` (Vue) all solve the same problem: letting a parent inject arbitrary content into a child's layout.
- Never do `props.name = 'X'` inside a component — treat the props object as read-only.
- TypeScript prop types (or `PropTypes` in plain React) document exactly what a component expects and catch misuse at compile time.

3. Composition Over Configuration

A common anti-pattern is building one large, configurable component with a growing list of boolean props (`showHeader`, `showFooter`, `isCompact`, `hasIcon...`) to handle every possible variation. Composition — building small components and combining them via `children`/slots — usually scales better: instead of a `Card` with ten boolean flags, build a plain `Card` that renders whatever children it's given, plus optional `CardHeader`/`CardFooter` sub-components a consumer assembles as needed. This mirrors how HTML itself works (a `<select>` is composed of `<option>` children, not configured with a giant options prop) and tends to produce more flexible, more testable components.

JSX

```
// Configuration-heavy (harder to extend indefinitely)
<Card showHeader title="Stats" showFooter footerText="Updated today" compact />

// Composition-based (flexible, each piece independently reusable)
<Card>
  <Card.Header>Stats</Card.Header>
  <Card.Body>...</Card.Body>
  <Card.Footer>Updated today</Card.Footer>
</Card>
```

4. Lists, Keys, and Reconciliation

Rendering a list of components from an array (via `.map()` in React) requires a key prop on each item — a stable, unique identifier the framework uses to track which DOM node corresponds to which data item across re-renders. Using the array index as a key works only if the list never reorders, is never filtered, and never has items inserted/removed from the middle — otherwise React can match the wrong DOM node to the wrong data, causing stale form input values or broken animations tied to the wrong item. A stable, data-derived id (a database primary key, a UUID) is the correct key whenever the list can change order or length.

JSX

```
function TodoList({ todos }) {  
  return (  
    <ul>  
      {todos.map((todo) => (  
        <li key={todo.id}>{todo.text}</li> // key = todo.id, NOT the array index  
      ))}  
    </ul>  
  );  
}
```

5. Container/Presentational Separation

A useful, if not absolute, pattern for keeping components focused is separating 'container' components (concerned with fetching data, managing state, business logic) from 'presentational' components (concerned only with how things look, receiving all their data via props with no knowledge of where it came from). A presentational `UserList` that just renders a list of user objects it's handed can be reused anywhere — a dashboard, a search results page, a modal — regardless of how or where those users were fetched, while the container component handling the actual data-fetching logic stays isolated and swappable.

6. Component Architecture in Angular and Vue

Angular components declare their inputs with `@Input()` decorators and outputs (events emitted upward) with `@Output()` combined with `EventEmitter`, structurally mirroring React's props-down/callback-up pattern but with more built-in ceremony (decorators, explicit dependency injection). Vue's single-file components declare props via `defineProps()` and emit events upward via `defineEmits()` in the Composition API — again the same underlying contract, just different syntax. Recognizing this shared shape (declared inputs, emitted outputs, one-directional data flow, composition of small pieces) is what makes moving between these frameworks a matter of learning new syntax rather than relearning frontend architecture from scratch.

7. Organizing Components in a Real Project

As a component tree grows, folder structure starts to matter. A common, scalable convention groups files by feature/domain (a `/features/checkout` folder containing every component, hook, and style specific to checkout) rather than by file type (one giant `/components` folder, one giant `/hooks` folder) — feature-based organization keeps related code physically close together and makes it obvious what can be deleted if a feature is removed. Shared, generic UI primitives (`Button`, `Input`, `Modal`) that have no feature-specific logic typically live in a separate `shared/ui` directory used across features.

Common Pitfalls

- Mutating props directly inside a child component instead of treating them as read-only and lifting change requests up via a callback prop.
- Using the array index as a list key when the list can reorder, filter, or have items inserted/removed from the middle.
- Building a single giant component with a dozen boolean configuration props instead of composing smaller, focused pieces.
- Mixing data-fetching/business logic directly into deeply nested presentational components, making them impossible to reuse elsewhere.
- Prop-drilling data through five layers of components that don't use it themselves, just to reach one deeply nested child.
- Organizing an entire large app in one flat /components folder with no feature-based grouping, making it hard to find related files.

Key Takeaways

- Components are the unit of reuse in every modern framework — props/inputs flow down, events/callbacks flow up.
- Treat props as strictly read-only from the receiving component's side, always.
- Prefer composition (children/slots) over an ever-growing list of configuration props for flexible, reusable components.
- List keys must be stable and data-derived, not the array index, whenever the list's order or length can change.
- The React/Angular/Vue syntaxes differ, but the props-down/events-up, compose-small-pieces architecture is the same discipline underneath all three.

Learn More

React — Passing Props to a Component

<https://react.dev/learn/passing-props-to-a-component>

React — Rendering Lists

<https://react.dev/learn/rendering-lists>

Vue.js — Component Basics

<https://vuejs.org/guide/essentials/component-basics.html>